Михаил Разаков

Compiler Backend / Code Generation Engineer

Москва / удалённо · razakov.mikhail@outlook.com · Telegram · GitHub

ПРОФИЛЬ

Разрабатываю экспериментальные компоненты compiler backend на Rust, C++/LLVM и C#/.NET. В собственном Rust-проекте реализовал проверку SSA/CFG, register allocation, spills, phi lowering и генерацию SysV x86-64. Отдельный verifier проверяет назначения регистров, а native-исполнение сравнивается с эталонным IR-интерпретатором. В МЦСТ работал с LLVM LICM; сейчас занимаюсь символьными вычислениями в SharpChecker (ИСП РАН).

ТЕХНОЛОГИИ

Backend / Codegen: Rust, iced-x86, SSA/CFG, dominance, edge-specific liveness, linear scan, seeded simulated annealing, spills, phi lowering, SysV x86-64

Compiler / Runtime: C++23, LLVM 22/IR, APInt, IR transformations; C#, .NET, CIL/CLR, interpreter и backend execution

Проверка / Инструменты: Независимый assignment verifier, reference interpreter, differential testing, Cargo tests, Clippy, Python, Git, Linux, CMake

ОПЫТ

ИСП РАН · SharpChecker · октябрь 2026 — настоящее время

• Работаю над символьными вычислениями для статического анализа программ на C#/.NET.

МЦСТ · стажировка по разработке компиляторов · июль — август 2026

• Работал над экспериментальным LLVM LICM: проверял допустимость переноса инструкций с учётом структуры цикла, обращений к памяти и побочных эффектов; сравнивал поведение до/после с помощью Python.

ИНЖЕНЕРНЫЕ ПРОЕКТЫ

x86-64 Compiler Backend & Register Allocation

Реализовал backend для ограниченного i64 IR: SSA/CFG, liveness, linear-scan и seeded simulated-annealing allocation, spills, phi lowering и SysV x86-64. Независимый verifier обнаруживает конфликты регистров и stack slots; phi lowering обрабатывает циклы параллельных перемещений. Native-код исполняется в отдельном процессе и сравнивается с IR-интерпретатором. На коммите dd0693c локально пройдены 42/42 debug-тестов, cargo fmt и строгий Clippy; release-тесты локально не завершились из-за дисковой квоты.

LLVM Interprocedural Global IV Optimization

LLVM pass анализирует межпроцедурные эффекты и аффинную эволюцию выбранных глобальных целочисленных переменных, затем проверяет допустимость и выполняет консервативное IR-преобразование. На LLVM 22.1.8 пройдены 29/29 регрессионных сценариев, включая неподдерживаемые случаи, где преобразование отклоняется. Это не LLVM target backend.

UniversalToolchain / Wist2

Разрабатываю модульную инфраструктуру с Bytecode/AIR, интерпретатором и CIL backend. LanguagePlan фиксирует маршруты исполнения; тесты проверяют совпадение поддерживаемых результатов interpreter/CIL. Репозиторный манифест содержит 1324 ожидаемых тестовых прохождения — это не повторный полный тестовый прогон во время аудита.

ОБРАЗОВАНИЕ

НИУ ВШЭ — Программная инженерия

ДОСТИЖЕНИЯ

2026 · LangDev — UniversalToolchain: Доклад об архитектуре компиляторной инфраструктуры 9 октября 2026 года. — Запись доклада

2025–2026 · «Юниор» НИЯУ МИФИ: Дипломы I степени в секции информационных технологий в 2025 и 2026 годах.

2026 · Отбор МЦСТ — LLVM/C++: По переписке организатора: 5,0/5,0 и 104/104 официальных тестов среди 49 представленных решений.